

实验二：原生应用开发

实验项目基本信息

- 实验编号：d20301035102、d20301009202
- 学时分配：4 学时
- 实验类型：验证型
- 每组人数：6 人
- 对应课程目标：课程目标 1

核心步骤列表

实验概览

1. 需求分析阶段：编写登录功能需求文档（输入验证规则、页面跳转逻辑）
2. AI 辅助编码阶段：使用 TRAE 生成登录页面骨架代码并完善业务逻辑
3. 测试验证阶段：编写输入验证测试用例，验证边界条件
4. 部署运维阶段：打包调试版本，记录平台差异对比报告

开发任务清单

任务阶段	主要内容	关键技术点
Android 登录	Java + Activity 跳转	EditText、Intent、Bundle
组内对比	技术分享	架构设计分析

✓ 成功标准

- ☐ Android 登录页面功能完整
- ☐ Activity 跳转与数据传递正常
- ☐ 输入验证逻辑正确
- ☐ 系统架构理解透彻
- ☐ 实验报告完成

实验任务与案例

核心任务

使用 Java 实现 Android 登录页面（EditText 输入验证、Activity 跳转与数据传递），对接 Spring Boot 后端接口实现完整的登录功能，包含 JWT 身份验证、用户信息本地存储等功能。

实战案例

开发"在线学习辅助平台"登录模块，用户（普通用户/儿童/家长）在登录页面输入账号和密码，系统验证成功后根据角色跳转到对应主页：儿童用户显示学习内容、课程推荐、学习任务，家长用户显示孩子学习数据、学习进度、成长报告，普通用户显示个性化学习中心、我的课程、通知公告等内容。

第一课时：Android 登录页面设计（4 学时）

2.1 需求分析阶段

5. 功能需求

- 账号输入（手机号/用户名）
- 密码输入
- 记住密码、自动登录选项
- 登录按钮、注册入口、忘记密码入口
- 输入验证（不能为空、长度限制、手机号格式验证）
- 对接后端登录接口
- JWT Token 存储与自动刷新
- 用户角色（普通用户/儿童/家长）识别

- 用户信息本地缓存
- 多角色权限控制

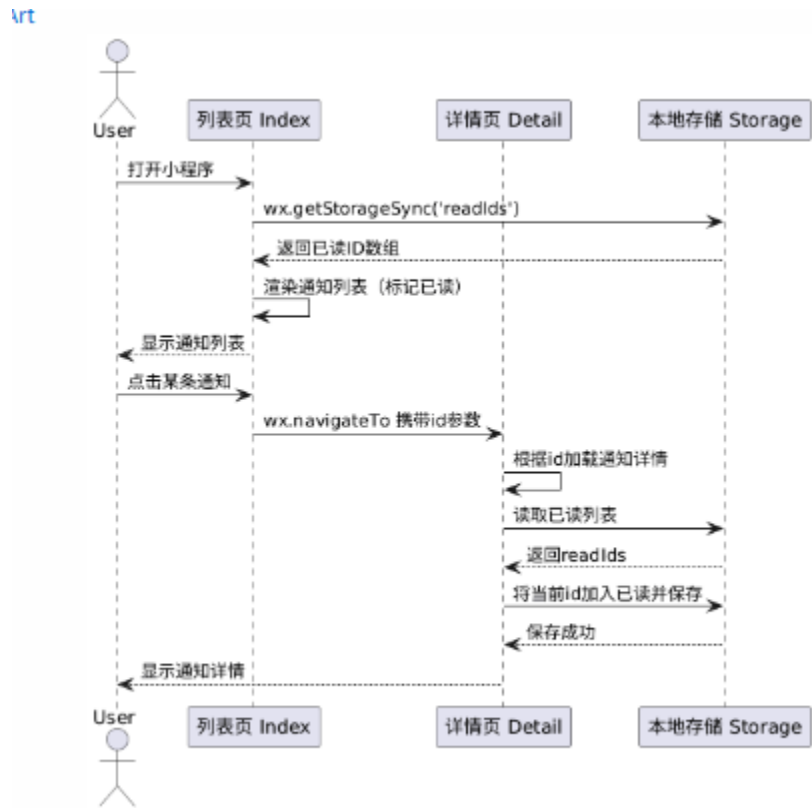
6. 界面需求

- Material Design 风格
- 输入框占位提示
- 密码隐藏显示切换
- 加载状态提示
- 错误信息友好提示
- 适合不同年龄层用户的交互设计

7. 跳转需求

- 登录成功 → 根据用户角色跳转对应主页
- 儿童用户：跳转到儿童学习主页
- 家长用户：跳转到家长管控主页
- 普通用户：跳转到学习中心主页
- 传递用户角色、ID、昵称等数据
- 登录状态持久化，下次启动自动校验

2.2 架构图及其分析



这张图展示了用户、列表页、详情页、本地存储之间，功能执行的交互时序与数据流向。

核心流程拆解：

用户打开小程序：触发列表页 Index 的初始化。

列表页读取已读状态：

列表页调用 `wx.getStorageSync('readIds')`，从本地存储获取已读通知 ID 数组。

本地存储返回 `readIds` 后，列表页渲染通知列表，并根据 ID 数组标记哪些通知为 “已读”，展示给用户。

用户点击通知跳转详情页：

列表页调用 `wx.navigateTo`，携带通知 ID 跳转到详情页 Detail。

详情页处理与标记已读：

详情页根据传入的 ID 加载通知详情内容。

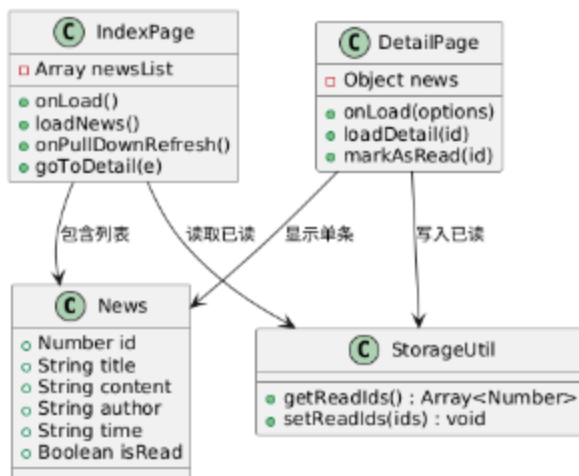
详情页读取本地存储的 `readIds`，将当前通知 ID 加入数组并保存回本地存储。

保存成功后，详情页将通知详情展示给用户。

关键信息：

清晰展示了谁在什么时候做了什么、数据怎么传递，是开发时梳理接口调用顺序的核心依据。

明确了「已读状态」的读写逻辑：列表页读、详情页写，保证数据一致性。



这张图定义了实现通知已读功能的 4 个核心类及其协作关系：

IndexPage（列表页）

属性：newsList 通知列表

方法：onLoad()、loadNews()、下拉刷新、跳转详情

作用：加载并渲染通知列表，通过 StorageUtil 标记已读状态

DetailPage（详情页）

属性：news 当前通知详情

方法：onLoad()、loadDetail()、markAsRead()

作用：加载通知详情，并调用 StorageUtil 标记为已读

News（通知模型）

属性：id、标题、内容、作者、时间、isRead 已读状态

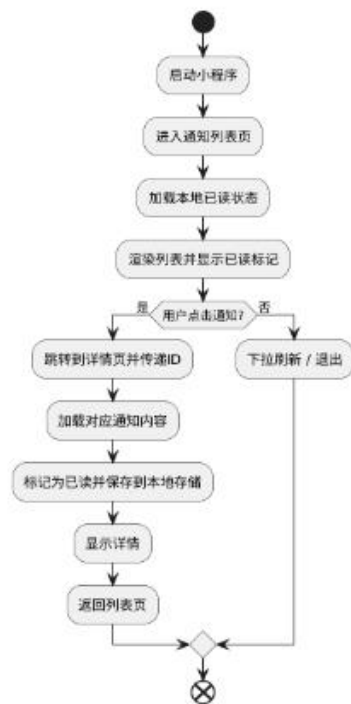
作用：定义单条通知的数据结构

StorageUtil（存储工具）

方法：getReadIds()、setReadIds()

作用：封装本地已读 ID 的读写逻辑，供列表页和详情页调用

协作逻辑：列表页用 StorageUtil 读取已读 ID，标记列表状态；用户点击后跳转到详情页，详情页加载内容并调用 StorageUtil 写入已读 ID，完成状态更新。



这张图展示了用户使用该功能时，完整的业务流程和分支逻辑。

核心流程拆解：

启动小程序：进入通知列表页。

初始化列表：加载本地已读状态 → 渲染列表并显示已读标记。

分支判断：

分支 1：用户点击通知：

跳转到详情页并传递 ID。

加载通知内容。

标记为已读并保存到本地存储。

显示详情 → 用户返回列表页。

分支 2：用户不点击通知：

下拉刷新或直接退出小程序。

流程结束：所有分支最终都会汇合，完成一次用户交互。

关键信息：

完整覆盖了用户的核心操作路径，包括正常流程和分支场景。

明确了「已读标记」的触发时机：用户点击通知进入详情页后，才会标记为已读，避免误操作。

2.3 Android 项目创建

8. 打开 Android Studio
9. 创建 Empty Activity 项目
10. 选择 Java 语言
11. 最低 SDK: API 24

2.4 AI 辅助编码

12. 使用 TRAE 描述需求: Java 登录页面, EditText 输入验证, Activity 跳转, 后端接口对接
13. AI 生成代码片段

14. 手动完善业务逻辑

```
<!-- activity_login.xml -->
<LinearLayout
    android:orientation="vertical"
    android:padding="16dp">

    <EditText
        android:id="@+id/et_username"
        android:hint="请输入学号"
        android:inputType="text"/>

    <EditText
        android:id="@+id/et_password"
        android:hint="请输入密码"
        android:inputType="textPassword"/>

    <CheckBox
        android:id="@+id/cb_remember"
        android:text="记住密码"/>

    <CheckBox
        android:id="@+id/cb_auto_login"
        android:text="自动登录"/>

    <Button
        android:id="@+id/btn_login"
        android:text="登录"/>
</LinearLayout>

// LoginActivity.java
public class LoginActivity extends AppCompatActivity
{ private UserViewModel userViewModel;
  private EditText etUsername;
  private EditText etPassword;
  private CheckBox cbAutoLogin;
  private Button btnLogin;
  private TextView tvForgotPassword;
  private TextView tvRegister;

  @Override
```



```

protected void onCreate(Bundle savedInstanceState)
{
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_login);

    userModel = new
    ViewModelProvider(this).get(UserViewModel.class);

    etUsername = findViewById(R.id.et_username);
    etPassword = findViewById(R.id.et_password);
    cbAutoLogin = findViewById(R.id.cb_auto_login);
    btnLogin = findViewById(R.id.btn_login);
    tvForgotPassword = findViewById(R.id.tv_forgot_password);
    tvRegister = findViewById(R.id.tv_register);

    btnLogin.setOnClickListener(v -> login());
    tvRegister.setOnClickListener(v -> startActivity(new
    Intent(this, RegisterActivity.class)));
    tvForgotPassword.setOnClickListener(v ->
    Toast.makeText(getApplicationContext(), "忘记密码功能开发中",
    Toast.LENGTH_SHORT).show());

    checkAutoLogin();
    observeLoginResult();
}

private void login() {
    String username = etUsername.getText().toString().trim();
    String password = etPassword.getText().toString().trim();

    if (TextUtils.isEmpty(username))
    {
        Toast.makeText(this, "请输入用户名",
        Toast.LENGTH_SHORT).show();
        return;
    }
    if (TextUtils.isEmpty(password))
    {
        Toast.makeText(this, "请输入密码",
        Toast.LENGTH_SHORT).show();
        return;
    }
    if (password.length() < 6)
    {
        Toast.makeText(this, "密码长度不能少于 6 位",
        Toast.LENGTH_SHORT).show();
        return;
    }

    btnLogin.setEnabled(false);
    userModel.login(username, password);
}

```

```

    }

    private void checkAutoLogin()
    {
        boolean autoLogin =
        SharedPreferencesUtil.getBoolean(AppConstants.SP_KEY_AUTO_LOGIN,
        false);

        if (autoLogin) {
            String savedUsername =
            SharedPreferencesUtil.getString(AppConstants.SP_KEY_USERNAME, "");
            String savedPassword =
            SharedPreferencesUtil.getString(AppConstants.SP_KEY_PASSWORD, "");
            if (!TextUtils.isEmpty(savedUsername)
            && !TextUtils.isEmpty(savedPassword)) {
                etUsername.setText(savedUsername);
                etPassword.setText(savedPassword);
                cbAutoLogin.setChecked(true);
                login();
            }
        }
    }

    private void observeLoginResult()
    {
        userViewModel.getLoginResult().observe(this, result -> {
            btnLogin.setEnabled(true);
            if (result != null && result.isSuccess())
            {
                if (cbAutoLogin.isChecked()) {

                    SharedPreferencesUtil.putBoolean(AppConstants.SP_KEY_AUTO_LOGIN,
                    true);

                    SharedPreferencesUtil.putString(AppConstants.SP_KEY_PASSWORD,
                    etPassword.getText().toString().trim());
                } else {

                    SharedPreferencesUtil.putBoolean(AppConstants.SP_KEY_AUTO_LOGIN,
                    false);

                    SharedPreferencesUtil.remove(AppConstants.SP_KEY_PASSWORD);
                }
                Toast.makeText(getApplicationContext(), "登录成功",
                Toast.LENGTH_SHORT).show();
                startActivity(new Intent(this,
                MainActivity.class));
                finish();
            } else {
                Toast.makeText(getApplicationContext(), result !=
                null ? result.getMessage() : "登录失败",
                Toast.LENGTH_SHORT).show();
            }
        });
    }

```

```
    }  
    });  
}  
}
```

2.5 创建首页页面

15. 新建 Activity: MainActivity

16. 布局显示欢迎信息和用户资料

```
// MainActivity.java  
public class MainActivity extends AppCompatActivity  
{ @Override  
    protected void onCreate(Bundle savedInstanceState)  
    { super.onCreate(savedInstanceState);  
      setContentView(R.layout.activity_main);  
  
      String username =  
      SharedPreferencesUtil.getString(AppConstants.SP_KEY_USERNAME, "");  
      TextView tvWelcome = findViewById(R.id.tv_welcome);  
      tvWelcome.setText("欢迎回来, " + username);  
  
      // 加载用户头像等其他信息  
    }  
}
```

2.6 测试验证阶段

17. 输入测试

- 空用户名测试
- 空密码测试
- 密码长度不足测试
- 正确凭证测试

错误凭证测



- 修改密码 →
- 绑定/修改手机号 →
- 绑定/修改邮箱 →
- 隐私设置 →

密码修改成功



- 修改密码 →
- 绑定/修改手机号 →
- 绑定/修改邮箱 →
- 隐私设置 →

手机号修改成功



忘记密码

请输入您的注册邮箱，我们将发送验证码到您的邮箱

发送验证码

[返回登录](#)

13:37

1台

97%



重置密码

请输入验证码和新密码

2453233917@qq.com

请输入验证码

请输入新密码

请确认新密码

验证码已发送，请查收邮件

返回登录

在线学习辅助平台

密码重置验证码

您好,

您正在重置密码, 您的验证码是:

007218

验证码有效期为5分钟, 请尽快使用。

如果这不是您本人操作, 请忽略此邮件。

13:37

1台

97%



请输入用户名

请输入密码

👁

☐ 自动登录

登录

忘记密码?

注册新账号

 密码重置成功, 请使用新密码登录

总结与思考

实验总结

- 掌握 Android Java 登录页面开发
- 理解前后端分离架构设计
- 学会 Activity 间数据传递和本地存储
- 掌握 JWT 身份验证流程
- 完成系统架构分析和设计理解

课后思考

18. 原生开发相比跨平台的优势和劣势
19. 如何设计更安全的用户身份验证机制
20. AI 辅助代码生成对开发效率的影响